

Relazione di Progetto:
“Kiki’s Store”

Leonardo Galimi
Hafsa Nouij

29 settembre 2026

Indice

1	Analisi	2
1.1	Descrizione e requisiti	2
1.1.1	Requisiti funzionali	2
1.1.2	Requisiti non funzionali	3
1.2	Modello del Dominio	4
2	Design	6
2.1	Architettura	6
2.2	Design dettagliato	8
2.2.1	Leonardo Galimi	8
2.2.2	Hafsa Nouij	11
3	Sviluppo	16
3.1	Testing automatizzato	16
3.1.1	Leonardo Galimi	16
3.1.2	Hafsa Nouij	17
3.2	Note di sviluppo	18
3.2.1	Leonardo Galimi	18
3.2.2	Hafsa Nouij	19
4	Commenti finali	21
4.1	Autovalutazione e lavori futuri	21
4.1.1	Leonardo Galimi	21
4.1.2	Hafsa Nouij	22
4.2	Difficoltà incontrate e commenti per i docenti	23
4.2.1	Leonardo Galimi	23
4.2.2	Hafsa Nouij	23
A	Guida utente	24
A.1	Inizio	24
A.2	Controlli di gioco	25
A.3	Dinamiche di gioco	25
A.3.1	Villaggio ed Esplorazione	25
A.3.2	Inventario e ricettario	26
A.3.3	Crafting	27
A.3.4	Minigame	27

Capitolo 1

Analisi

1.1 Descrizione e requisiti

Kiki's Store è un videogioco d'avventura in due dimensioni con grafica in pixel art, ispirato al film d'animazione *Kiki's Delivery Service*. L'utente controlla una giovane apprendista strega che ha il compito di ambientarsi in una cittadina fantastica e avviare la propria attività: una bottega di pozioni magiche.

L'esperienza di gioco unisce diverse meccaniche collegate tra loro:

- **Esplorazione e raccolta:** il giocatore esplora liberamente le strade del villaggio, muovendosi all'interno di una mappa bidimensionale. Durante l'esplorazione è possibile individuare e raccogliere ingredienti comparsi a terra.
- **Gestione del personaggio:** Le attività della protagonista consumano la sua energia, che quindi dev'essere gestita con attenzione insieme alle monete guadagnate.
- **Alchimia e creazione (Crafting):** all'interno della bottega, la protagonista può consultare un ricettario per combinare gli ingredienti posseduti. Unendo le componenti richieste dalla ricetta è possibile distillare nuove pozioni.
- **Minigiochi integrati:** Il gioco introduce sfide secondarie con regole dedicate, come una prova di volo su scopa a scorrimento, un minigioco di memoria basato sul riconoscimento delle pozioni.

1.1.1 Requisiti funzionali

- **Movimento e controllo:** il sistema deve permettere al giocatore di muovere il personaggio nelle quattro direzioni cardinali (su, giù, sinistra, destra) all'interno dell'ambiente di gioco.
- **Rilevamento ostacoli:** il personaggio non deve poter attraversare elementi solidi della mappa come muri, pareti degli edifici o confini dell'area esplorabile.
- **Generazione e raccolta risorse:** il sistema deve far comparire periodicamente sulla mappa nuovi ingredienti posizionandoli unicamente su spazi calpestabili; il passaggio del personaggio sopra di essi deve permetterne la raccolta, inserendoli nell'inventario e rimuovendoli dal terreno.

- **Gestione dell'inventario:** il sistema deve tenere traccia degli oggetti raccolti e delle pozioni create, memorizzandone nome, tipologia e quantità aggiornate.
- **Consultazione del ricettario:** il giocatore deve poter aprire un ricettario per visionare le pozioni realizzabili e i requisiti in ingredienti necessari per ciascuna di esse.
- **Sistema di creazione (Crafting):** il sistema deve permettere la selezione degli ingredienti e la preparazione delle pozioni, se gli ingredienti selezionati corrispondono a una ricetta valida, questi vengono consumati dall'inventario per produrre la pozione finita.
- **Gestione dell'energia e delle monete:** il sistema deve visualizzare a schermo l'energia della protagonista e le monete, scalando l'energia dopo determinate attività e aggiornando le monete quando necessario.
- **Transizione e gestione degli scenari:** il sistema deve consentire il passaggio tra la schermata principale di esplorazione, la schermata d'inventario e le aree interne o dei minigiochi, congelando lo stato del mondo primario per riprenderlo senza alterazioni al rientro.
- **Svolgimento dei minigiochi:**
 - *Volo con la scopa:* una sessione a scorrimento in cui il giocatore guida la strega in volo schivando gli ostacoli aerei.
 - *Memory delle pozioni:* una sfida basata sulla memorizzazione e sull'accoppiamento di tessere raffiguranti le pozioni.
- **Interazione con il mondo:** il sistema deve riconoscere punti di teletrasporto sulla mappa per consentire l'ingresso in strutture o l'inizio di un minigioco.

1.1.2 Requisiti non funzionali

- **Portabilità:** il gioco deve poter essere avviato ed eseguito senza modifiche sui principali sistemi operativi (Windows, macOS, Linux).
- **Fluidità e reattività:** la risposta ai comandi di movimento impartiti dall'utente deve essere immediata, garantendo un aggiornamento a schermo continuo e senza rallentamenti evidenti durante la transizione tra le schermate.
- **Consistenza dei dati di gioco:** il passaggio verso un minigioco o un altro stato non deve causare la perdita di coordinate, lo svuotamento dell'inventario o la scomparsa degli oggetti presenti a terra.

1.2 Modello del Dominio

Il dominio del gioco modella le regole e i concetti principali dell'esperienza di gioco. Il sistema descrive una sessione in cui l'utente esplora il villaggio, raccoglie risorse comparse sul terreno e crea pozioni magiche attraverso la combinazione di ingredienti.

Dall'analisi del problema emergono le seguenti entità concettuali primarie:

- **Giocatore (Player):** rappresenta l'avatar controllato dall'utente all'interno del mondo di gioco. Tiene traccia della posizione, della direzione verso cui è rivolto, dell'energia disponibile per compiere azioni e della scorta di monete accumulate.
- **Mappa (TileMap):** definisce l'ambiente bidimensionale strutturato a griglia di celle. Distingue le superfici calpestabili dagli ostacoli solidi invalicabili e identifica i punti di interazione o di passaggio verso altre aree.
- **Oggetto a Terra (GroundItem):** rappresenta la manifestazione fisica di una risorsa collocata sulla mappa. Associa un ingrediente alle proprie coordinate nel mondo, consentendone la raccolta automatica al passaggio del giocatore.
- **Ingrediente (Ingredient):** costituisce la materia prima naturale raccolta nel mondo, definita da un nome, dalla quantità disponibile e da una tipologia (fiori, piante o radici).
- **Pozione (Potion):** rappresenta il preparato ottenuto tramite la combinazione di risorse. È definita da una descrizione e dall'effetto prodotto sul consumatore.
- **Inventario (Inventory):** serve come archivio delle risorse ottenute dal giocatore. Gestisce e organizza le scorte di ingredienti raccolti e di pozioni create, permettendo di visualizzare, depositare e consumare gli oggetti a disposizione.
- **Ricetta (Recipe):** descrive la combinazione di ingredienti necessaria per creare una pozione e tiene traccia se è già stata scoperta durante la partita.

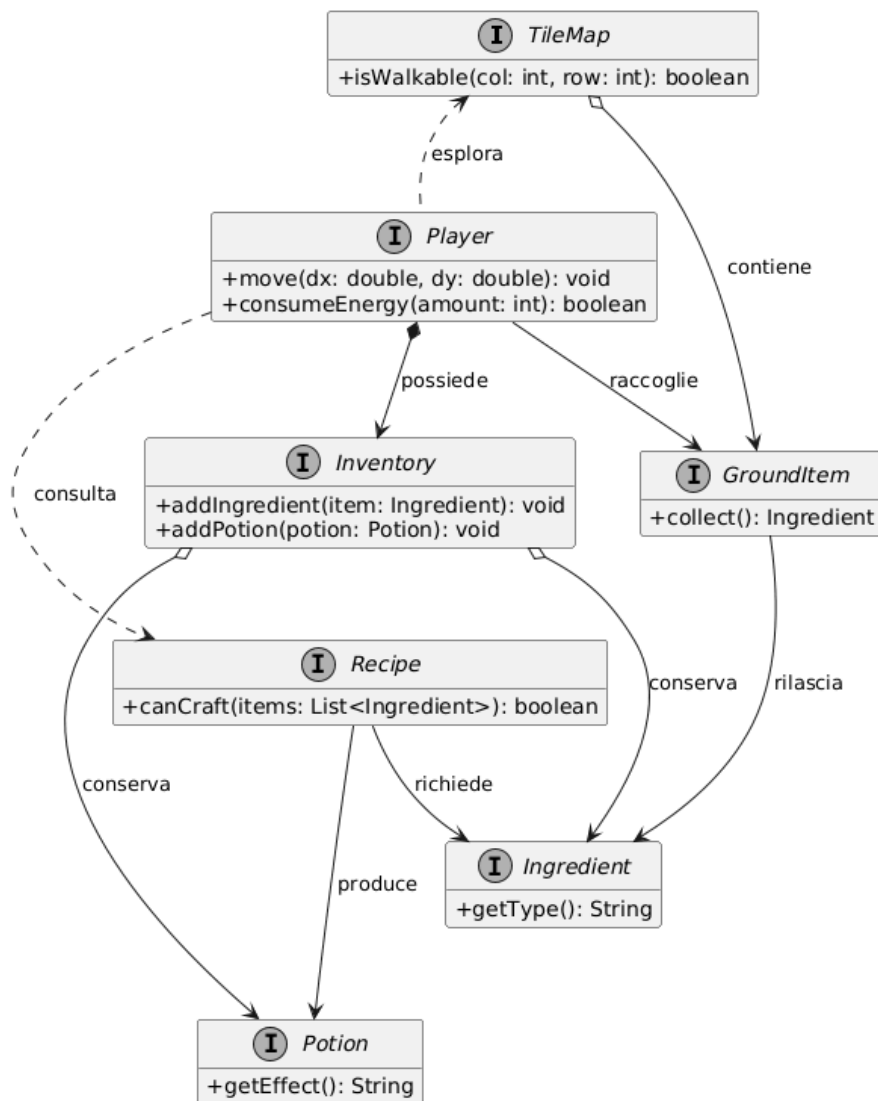


Figura 1.1: Diagramma concettuale del modello di dominio di Kiki's Store.

Capitolo 2

Design

2.1 Architettura

Il progetto è impostato sul pattern **Model-View-Controller (MVC)**, affiancato a un **Game Loop** per gestire il tempo e uno **State Pattern** su pila (`Deque<GameState>`) per organizzare i vari stati di gioco.

Questa impostazione garantisce una chiara separazione tra la logica del gioco e la sua visualizzazione a schermo:

- **Engine e Ciclo di Vita:** gestisce il tempo di gioco e il passaggio tra le schermate. `GameEngine` usa l'`AnimationTimer` di JavaFX per mandare avanti il Game Loop, eseguendo a ogni frame i tre passaggi: aggiornamento della logica, pulizia dello schermo e ridisegno (*update* → *clear* → *render*). Il controllo passa poi a `GameStateManager`, che gestisce le schermate tramite una pila: quando si entra in una nuova sezione (come un minigioco o il ricettario), questa viene inserita in cima con `pushState`, mettendo in pausa quella precedente con `pause()`; quando la si chiude, viene tolta con `popState` e la schermata sottostante riparte con `resume()`.
- **Controller:** fa da tramite tra i comandi dell'utente e lo stato del gioco. Attraverso `InputHandler` intercetta i tasti premuti sulla finestra JavaFX in modo indipendente dall'hardware, permettendo ai controller (come `PlayerController` per il movimento) di leggere gli input attivi e aggiornare il modello.
- **Model:** racchiude i dati e le regole logiche del gioco. Gestisce sia l'ambiente di simulazione (il giocatore `Player`, la mappa e le collisioni) sia le strutture dell'inventario, delle ricette e degli oggetti. Lo stato interno viene modificato unicamente dai controller e non mantiene alcun riferimento verso la vista.
- **View:** ha un ruolo passivo e si limita a disegnare su un `Canvas`. Per impedire che l'interfaccia possa alterare accidentalmente i dati di gioco, la vista non accede direttamente alle classi del modello, ma riceve proiezioni di sola lettura sotto forma di record Java immutabili (`RenderData`). I componenti di rendering (`ViewRenderers`) leggono questi record per disegnare a schermo entità, elementi dello scenario e interfaccia utente.

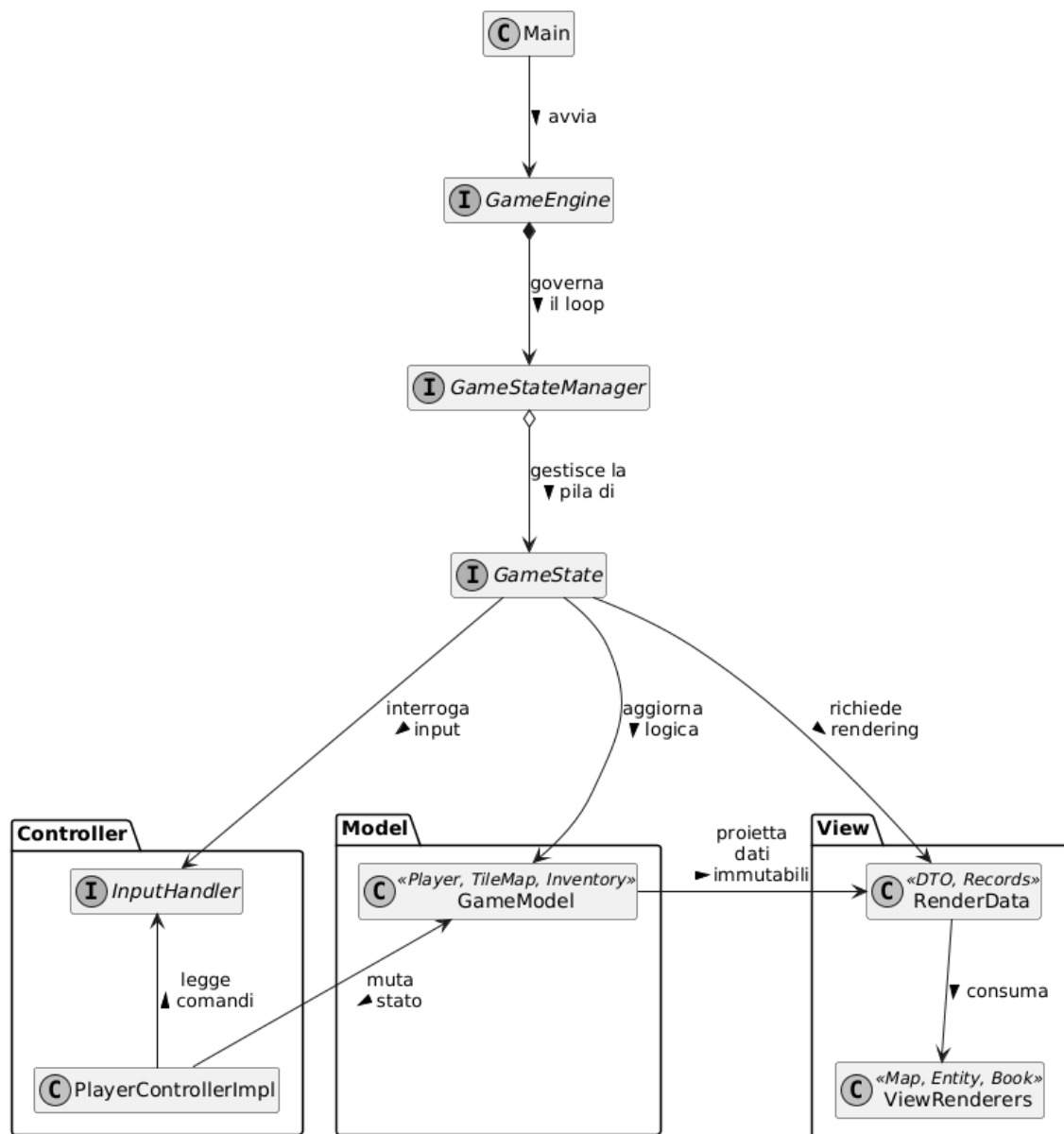


Figura 2.1: Diagramma architetturale complessivo di Kiki's Store.

2.2 Design dettagliato

2.2.1 Leonardo Galimi

Gestione del ciclo vitale e transizioni tra contesti (State Pattern su Stack)

Problema Il gioco richiede la coesistenza di diversi contesti operativi: la navigazione della mappa primaria, l'inventario e i minigiochi. In più, la transizione verso un minigioco o un menu non deve distruggere lo stato corrente, ma congelarlo per poi riprendere esattamente dallo stesso punto.

Soluzione Il problema è stato risolto usando lo **State Pattern** esteso con una struttura a pila, incapsulata nella classe **GameStateManagerImpl** e gestita dall'interfaccia **GameStateTransition**.

Ciascun contesto del gioco implementa l'interfaccia **GameState**, fornendo i metodi per gestire le varie fasi di ciclo vitale **init()**, **update()**, **render()**, **pause()** e **resume()**:

- L'invocazione di **pushState(newState)** sospende lo stato corrente invocandone il metodo **pause()** e colloca il nuovo stato (ad esempio **MinigameFlyState**) in cima alla pila.
- L'invocazione di **popState()** rimuove lo stato terminato e risveglia lo stato sottostante usando **resume()**, ripristinando collisioni e riferimenti del giocatore senza ricaricare le risorse dal disco.
- Il motore grafico (**GameEngineImpl**) manda le chiamate di aggiornamento e disegno solamente allo stato in cima (**peek()**).

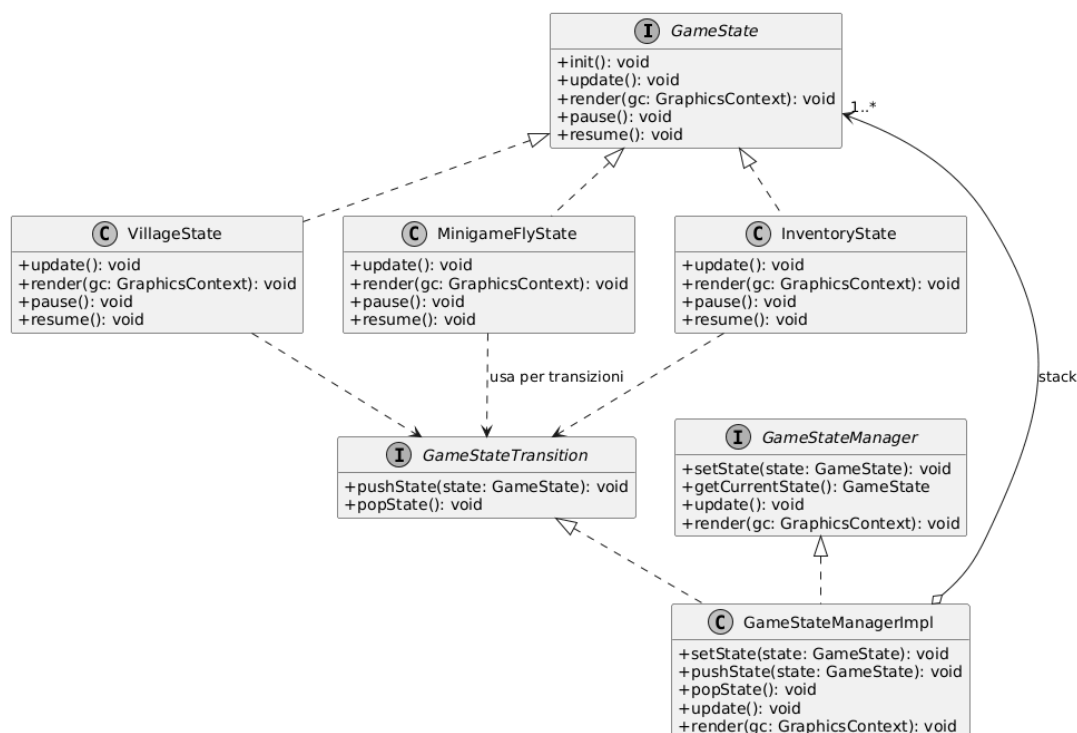


Figura 2.2: Diagramma UML dell'architettura a stati basata su State Pattern e Stack.

Movimento e gestione delle collisioni asse per asse

Problema Il personaggio si muove in pixel in modo continuo, mentre il mondo è composto da una griglia di caselle (**GameTile**) con comportamenti diversi: zone calpestabili, muri non traversabili e caselle evento. Controllare lo spostamento diagonale (dx, dy) tutto insieme creava un problema: bastava sfiorare un angolo o una parete per bloccare completamente il personaggio, impedendogli di scivolare lungo i muri e rendendo i comandi fastidiosi e difficili da usare.

Soluzione Ho affidato il controllo dei movimenti alla classe **CollisionHandler**, che viene passata direttamente al modello del giocatore per separare la logica del movimento da quella della mappa.

Per rendere lo spostamento più fluido, il controllo viene fatto in due momenti separati:

1. Si prova prima il movimento in orizzontale (asse X). Se i quattro angoli della hitbox del personaggio non toccano ostacoli, la coordinata X viene aggiornata.
2. Subito dopo si fa la stessa verifica in verticale (asse Y). Se la via è libera, si aggiorna anche la coordinata Y.

In questo modo, se ad esempio Kiki cammina in diagonale contro un muro verticale, la coordinata X si ferma contro l'ostacolo ma la coordinata Y continua a salire o scendere, permettendole di scivolare lungo la parete senza bloccarsi. Inoltre, il metodo `getInteractableTileId()` controlla se il personaggio si trova sopra a caselle speciali, avvisando lo stato di gioco quando è il momento di attivare teletrasporti o minigiochi.

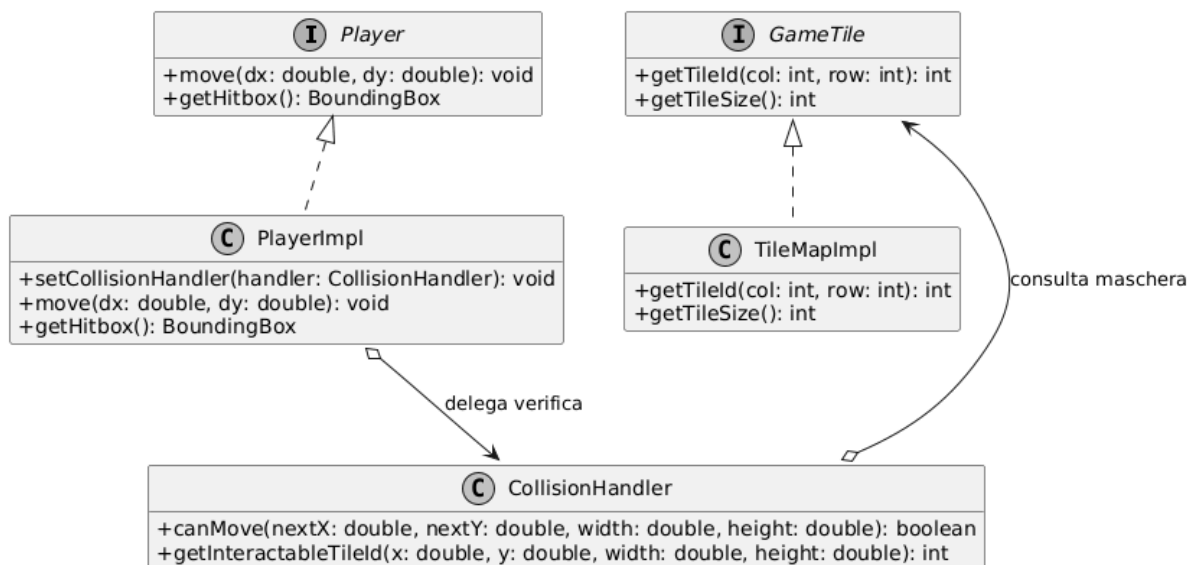


Figura 2.3: Diagramma UML dell'integrazione tra Player, CollisionHandler e TileMap.

Generazione autonoma e raccolta delle risorse nel mondo

Problema Il sistema di gioco ha bisogno della presenza costante di ingredienti sparsi per la mappa. Posizionarli casualmente senza vincoli comporterebbe però dei problemi: elementi generati all'interno di ostacoli o edifici, sovrapposizioni su celle già occupate e una generazione così numerosa da saturare la memoria. In più, la rimozione degli elementi raccolti durante il ciclo di aggiornamento rischia di generare eccezioni di concorrenza (`ConcurrentModificationException`).

Soluzione La gestione di questo meccanismo avviene nella classe `ItemSpawnerImpl`, che gestisce il ciclo di vita degli oggetti nel mondo:

- **Verifica dei vincoli:** a intervalli regolari di tick, il generatore inserisce un numero controllato di elementi fino a raggiungere la capienza massima. Tramite l'interrogazione della maschera di collisione (`GameTile`), lo spawn avviene solo su celle calpestabili (ID 0) e traccia le caselle già assegnate tramite un insieme di coordinate (`Set<GridPos>`) per evitare sovrapposizioni.
- **Raccolta e rimozione sicura:** il metodo `checkCollection()` calcola l'intersezione tra la hitbox del giocatore e quella dei singoli `GroundItem` con il bounding box. La rimozione degli elementi raccolti dalla lista attiva avviene in sicurezza tramite un `Iterator` esplicito, evitando conflitti durante lo scorrimento della collezione.
- **Incapsulamento e protezione:** la lista restituita da `getActiveItems()` è protetta tramite `Collections.unmodifiableList`, così che dall'esterno possa essere unicamente letta e non modificata.

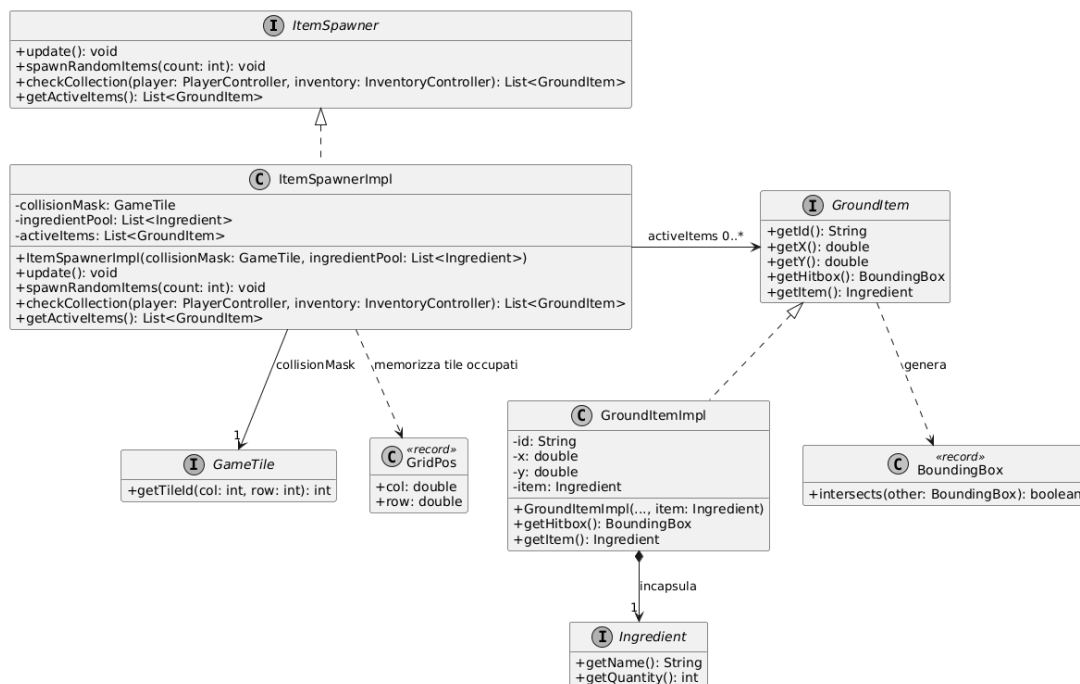


Figura 2.4: Diagramma UML del sottosistema di generazione e gestione del ciclo vitale degli oggetti.

2.2.2 Hafsa Nouij

Consultazione dell'inventario e del ricettario (Strategy)

Problema Il giocatore deve poter consultare, in un'unica schermata a forma di libro, sia gli oggetti posseduti sia le ricette già scoperte. Le due pagine hanno contenuti e interazioni completamente diversi (una griglia di caselle da una parte, le schede delle ricette sfogliabili dall'altra), ma condividono tutto il resto: l'animazione di apertura e chiusura, il cambio pagina e i segnalibri per passare da una sezione all'altra.

Soluzione Il contenuto di ogni sezione è stato separato dalla gestione del libro tramite l'interfaccia `BookSection`, implementata da `InventorySection` e `RecipeSection`. `BookState` gestisce solo ciò che è comune (animazioni, segnalibri, input di navigazione) e delega il disegno e l'aggiornamento alla sezione attiva, senza conoscerne il tipo concreto (Figura 2.5).

Un'alternativa sarebbe stata gestire entrambe le sezioni dentro `BookState`, distinguendole con uno `switch` nei metodi di disegno e di aggiornamento. Sarebbe stata più immediata, ma avrebbe concentrato in una sola classe la logica di due schermate diverse, e ogni nuova sezione avrebbe richiesto di modificarla in più punti. Con la soluzione adottata, aggiungere una sezione significa scrivere una nuova implementazione di `BookSection` e collegarla a un segnalibro: il libro prevede già un terzo segnalibro, dedicato agli ordini dei clienti, per ora disattivato e predisposto per un'estensione futura.

La soluzione applica il pattern **Strategy**: `BookSection` è l'interfaccia della strategia, `InventorySection` e `RecipeSection` sono le strategie concrete e `BookState` è il contesto, che le usa in modo trasparente.

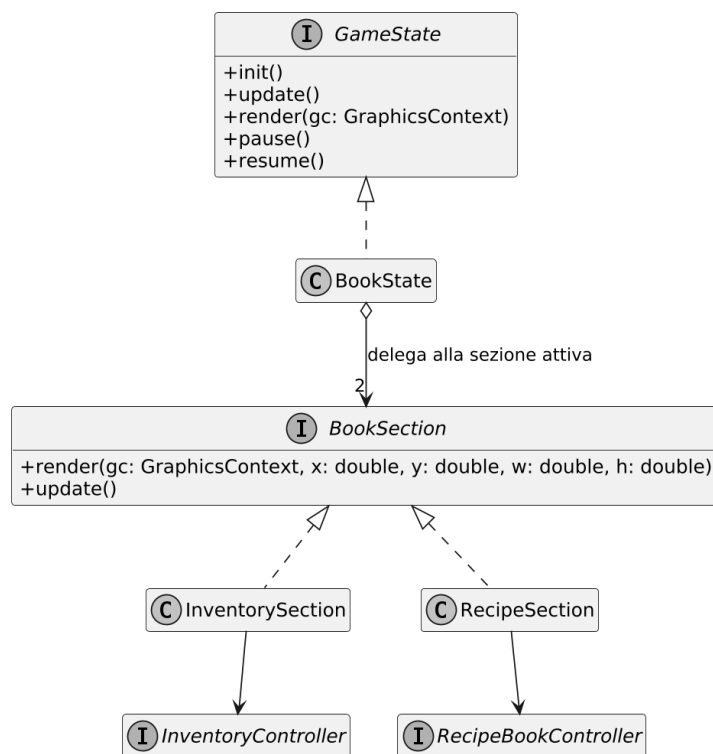


Figura 2.5: Il pattern Strategy applicato alle sezioni del libro.

Gestione uniforme di ingredienti e pozioni

Problema L'inventario contiene due tipi di oggetti, ingredienti e pozioni, sui quali si eseguono le stesse operazioni: aggiungere una quantità (sommandola se l'oggetto è già presente, creandolo se non lo è e se c'è ancora spazio) e toglierne una (rimuovendo l'oggetto quando la quantità arriva a zero). Scrivere queste regole due volte, una per tipo, porta a duplicazione e al rischio che le due versioni si comportino in modo diverso. Inoltre le liste dell'inventario non devono poter essere modificate dall'esterno, aggirando queste regole.

Soluzione `Inventory` espone ingredienti e pozioni solo come liste non modificabili e offre metodi dedicati per inserirli e rimuoverli. Le regole sulle quantità sono scritte una sola volta in `InventoryControllerImpl`, in due metodi generici su `T extends GameItem` (Figura 2.6). Ciò che cambia tra ingredienti e pozioni viene passato come parametro: un `Supplier<T>` crea il nuovo oggetto, ed è invocato solo se l'oggetto non è già presente, mentre un `Consumer<T>` lo inserisce o lo rimuove dall'inventario (ad esempio `inventory::addIngredient`).

L'alternativa più semplice, restituire le liste interne e modificarle direttamente dal controller, avrebbe violato l'incapsulamento dell'inventario. Restano due limiti: gli oggetti contenuti nelle liste sono modificabili e le regole sulle quantità risiedono nel controller anziché nel model; una soluzione più pulita le sposterebbe in `Inventory`, rendendo gli oggetti immutabili.

Si tratta del pattern **Strategy** in forma funzionale: le strategie sono i `Supplier` e i `Consumer` passati ai metodi generici, realizzati con lambda e method reference invece che con classi dedicate.

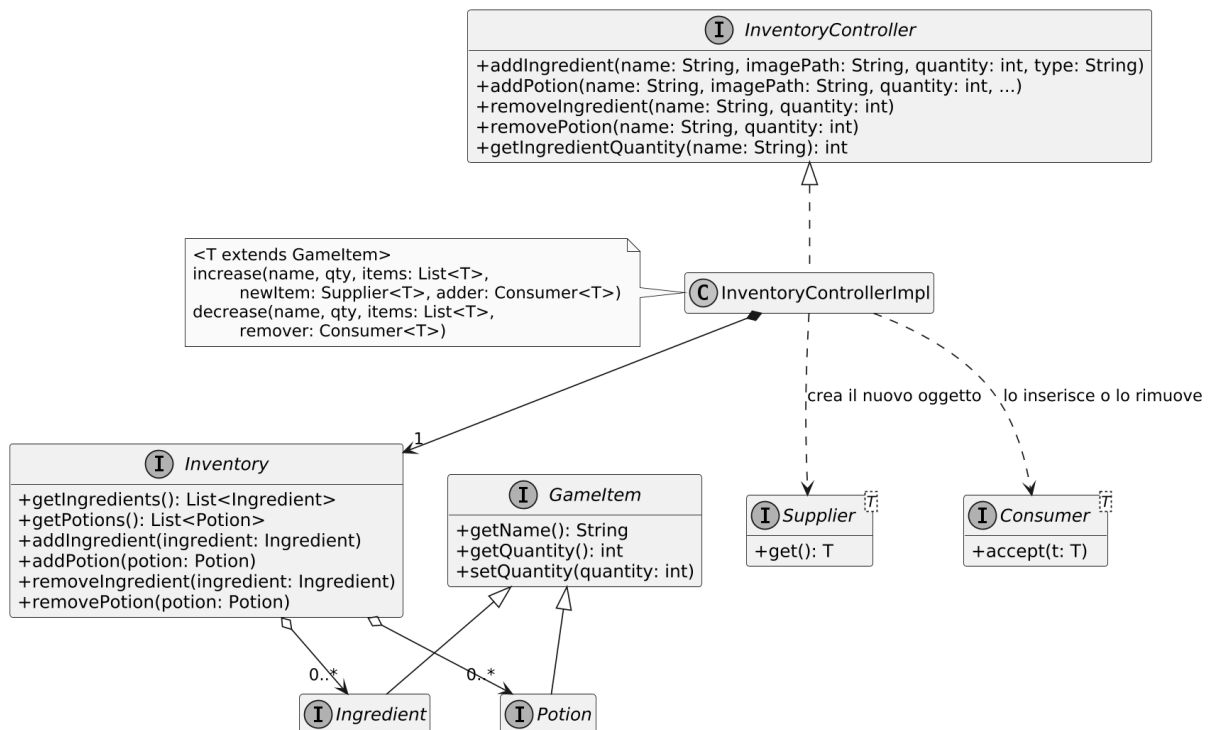


Figura 2.6: Gestione dell'inventario: le parti che variano tra ingredienti e pozioni vengono passate come interfacce funzionali.

Ricette e preparazione delle pozioni

Problema Preparare una pozione coinvolge più parti del gioco: bisogna riconoscere se gli ingredienti scelti corrispondono a una ricetta, consumarli dall'inventario, aggiungere la pozione ottenuta, sbloccare la ricetta nel ricettario e scalare l'energia della protagonista. Se la combinazione non è valida si ottiene una pozione fallita. Inoltre ricette, ingredienti e pozioni sono numerosi e devono poter essere modificati o ampliati senza intervenire sul codice.

Soluzione Ricette, ingredienti e pozioni sono descritti in file JSON e caricati all'avvio da `RecipeBookImpl` e `GameCatalogImpl`: aggiungere una pozione richiede solo di modificare i dati. Ogni ricetta elenca gli ingredienti richiesti con la quantità necessaria, e il riconoscimento della ricetta non dipende dall'ordine in cui gli ingredienti sono stati selezionati.

La preparazione è coordinata da `CraftingControllerImpl`, che non contiene la logica di inventario né quella del ricettario, ma le compone: riceve nel costruttore un `InventoryController` e un `RecipeBookController` e dipende solo dalle loro interfacce (Figura 2.7). In caso di successo consuma gli ingredienti indicati dalla ricetta, aggiunge la pozione e sblocca la ricetta; in caso contrario consuma gli ingredienti selezionati e produce la pozione fallita. La schermata del calderone (`CraftingState`) raccoglie la selezione del giocatore, verifica tramite `PlayerController` che la protagonista abbia energia sufficiente e affida la preparazione al controller.

Un'alternativa sarebbe stata affidare la preparazione all'inventario, che però avrebbe dovuto conoscere il ricettario, mescolando due responsabilità. Ricevere i collaboratori dall'esterno permette inoltre di condividere lo stesso inventario con il libro e con il Memory, e di verificarne il comportamento nei test.

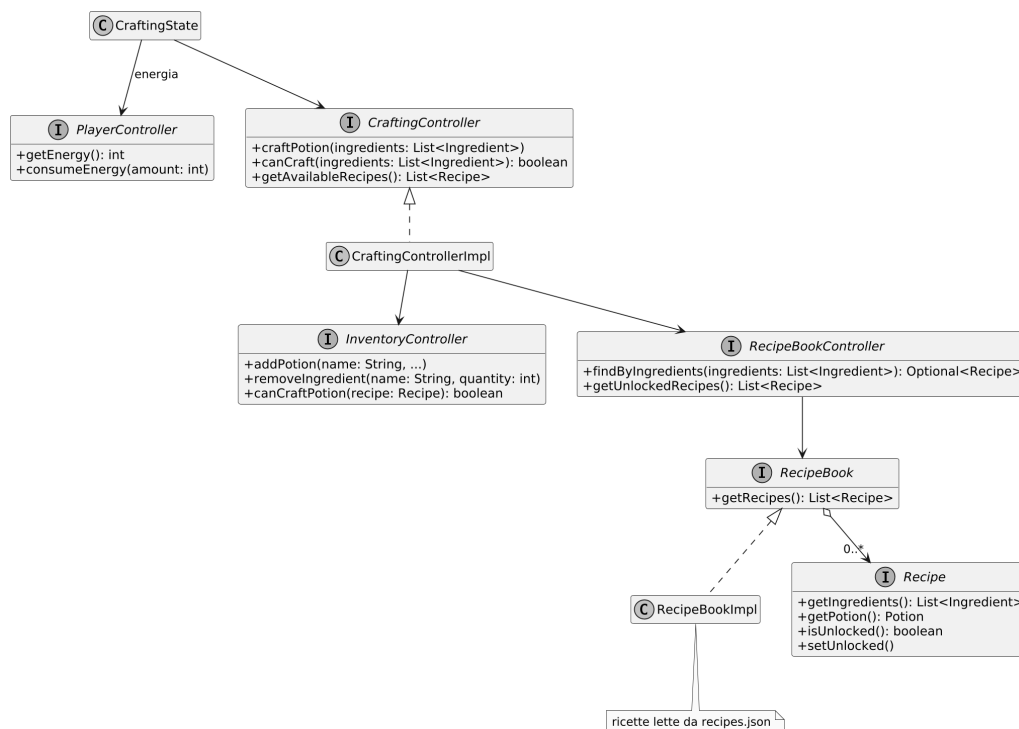


Figura 2.7: Il controller del crafting compone inventario e ricettario.

Minigioco del Memory

Problema Il Memory richiede di mescolare le carte, mostrarle brevemente all’inizio, gestire due carte scoperte alla volta con una pausa prima di ricoprirle in caso di errore, misurare tempo e mosse e, alla fine, assegnare una ricompensa che modifica l’energia, le monete o l’inventario della protagonista. Tutto questo va integrato nel ciclo di gioco, che aggiorna lo stato a ogni frame.

Soluzione Il minigioco è diviso in tre parti con responsabilità distinte (Figura 2.8). Il model (**MemoryBoard** e **MemoryCard**) rappresenta solo la griglia e lo stato di ogni carta; le carte raffigurano le pozioni del catalogo. **MemoryControllerImpl** contiene le regole: il mescolamento, il confronto delle coppie, il punteggio e la ricompensa. **MemoryState** disegna la griglia e traduce i clic del giocatore in chiamate a **flipCard**.

Lo svolgimento della partita è modellato come una macchina a stati finiti (Figura 2.9), avanzata a ogni chiamata di **update()**: i tempi di anteprima e di attesa sono contati in tick del ciclo di gioco, così la logica resta indipendente dalla visualizzazione.

Un’alternativa sarebbe stata gestire le fasi con più flag booleani indipendenti; una singola variabile di fase è preferibile perché rende impossibili combinazioni incoerenti, come una partita contemporaneamente in anteprima e in attesa dopo un errore.

La ricompensa viene assegnata tramite le interfacce **PlayerController**, **InventoryController** e **GameCatalog**, senza conoscerne le implementazioni: la parte sviluppata dall’altro componente del gruppo è raggiunta solo attraverso un contratto.

Per questo problema non è stato applicato un pattern specifico: con quattro fasi e transizioni semplici, una macchina a stati gestita all’interno del controller è stata ritenuta più leggibile di una gerarchia di classi.

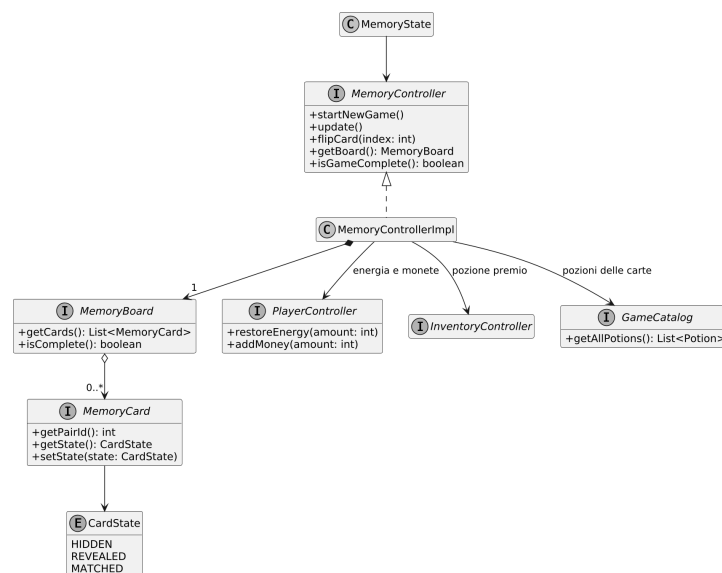


Figura 2.8: Separazione tra model, controller e view nel Memory.

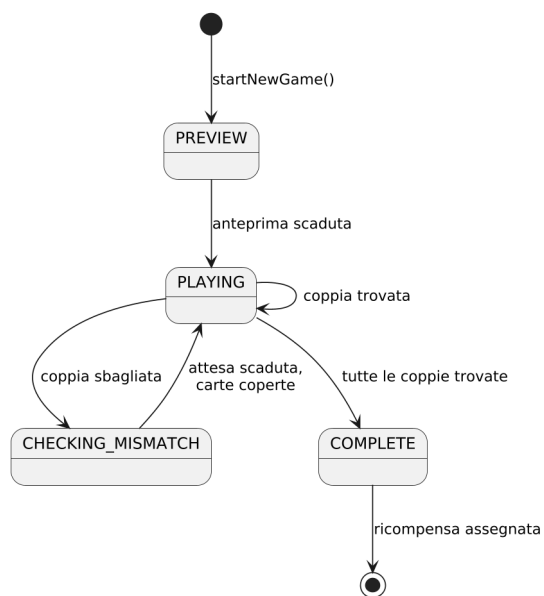


Figura 2.9: Le fasi di una partita di Memory.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

3.1.1 Leonardo Galimi

Per quanto riguarda la mia parte del progetto ho concentrato le attività di testing sul motore di transizione degli stati, sul modello del personaggio e sui sistemi di gestione del mondo e degli oggetti. Nello specifico ho eseguito test su:

- **Gestione degli stati (GameStateManager):** verifica dell'inizializzazione immediata dello stato all'inserimento su stack vuoto, controllo degli aggiornamenti esclusivamente allo stato attivo in cima alla pila, gestione del ciclo vitale con verifica dell'invocazione di `pause()` e `resume()` durante le operazioni di push e pop, e controllo del reset completo della pila tramite `setState()`.
- **Personaggio giocabile (Player):** controllo delle statistiche di partenza (risorse, posizione, orientamento e stato di animazione), aggiornamento manuale delle coordinate, verifica della gestione delle risorse con test sul decremento di energia, accumulo delle monete e rispetto del limite massimo di energia, oltre al controllo del movimento e aggiornamento dello stato in assenza di ostacoli.
- **Generazione e raccolta oggetti (ItemSpawner):** verifica del corretto posizionamento degli oggetti solo su celle calpestabili, controllo del rispetto del limite massimo di elementi generabili in base ai tile liberi disponibili, blocco dello spawn su mappe interamente solide, conferma della rimozione dal mondo e inserimento nell'inventario in caso di raccolta, e controllo dell'immutabilità della lista degli oggetti attivi esposta al modello.
- **Mappa logica e collisioni (TileMap e CollisionHandler):** controllo delle dimensioni in tile della matrice e gestione di griglie vuote, lettura corretta degli indici di cella con fallback su tile solido in caso di coordinate fuori mappa, verifica della traversabilità su spazi vuoti e blocco del movimento su ostacoli e limiti della mappa, e riconoscimento dei tile trigger interattivi garantendone contemporaneamente la traversabilità.
- **Oggetti a terra (GroundItem):** controllo delle proprietà base dell'entità a terra (identificatore, dimensioni e animazione), verifica dell'integrità dei dati dell'ingrediente racchiuso (Ingredient) e correttezza del calcolo del riquadro di collisione (hitbox) rispetto alla posizione nel mondo.

3.1.2 Hafsa Nouij

I test sulla mia parte del progetto sono scritti con JUnit 5, sono completamente automatici e coprono il model e i controller di inventario, ricettario, crafting e Memory, per un totale di 44 test. Nello specifico ho eseguito test su:

- **Oggetti di gioco (Ingredient, Potion, Recipe):** verifica dei valori impostati dai costruttori e dei setter, della marcatura delle pozioni fallite e dello stato iniziale delle ricette, che partono bloccate e vengono sbloccate con `setUnlocked()`.
- **Caricamento dei dati (GameCatalog e RecipeBook):** lettura di ingredienti, pozioni e ricette dai file JSON reali del gioco, controllo che tutte le ricette partano bloccate e che lo sblocco sposti una ricetta nella lista di quelle scoperte.
- **Inventario (Inventory e InventoryController):** aggiunta di ingredienti con somma delle quantità se già presenti, rimozione parziale e totale di ingredienti e pozioni (con eliminazione della voce quando la quantità arriva a zero), controllo delle quantità disponibili e individuazione degli ingredienti mancanti per una ricetta.
- **Ricettario (RecipeBookController):** riconoscimento di una ricetta indipendentemente dall'ordine degli ingredienti, risultato vuoto per combinazioni sconosciute, ricerca per nome parziale e senza distinzione tra maiuscole e minuscole, ricerca per effetto e aggiornamento del conteggio delle ricette sbloccate.
- **Crafting (CraftingController):** riconoscimento delle combinazioni valide e non valide, creazione della pozione con sblocco della ricetta, produzione della pozione fallita in caso di errore, consumo della sola quantità richiesta per ogni ingrediente anche quando se ne possiedono più unità, aggiunta di una sola pozione per preparazione ed esclusione delle ricette sbloccate ma non realizzabili per mancanza di ingredienti.
- **Memory (MemoryBoard, MemoryCard e MemoryController):** costruzione di una griglia completa con due carte per ogni coppia, stato iniziale coperto delle carte, riconoscimento del completamento della partita, marcatura delle coppie trovate e copertura delle carte sbagliate dopo il tempo di attesa. Le parti che dipendono dal giocatore sono sostituite nei test da un oggetto fittizio.

3.2 Note di sviluppo

3.2.1 Leonardo Galimi

Vengono elencate di seguito le funzionalità avanzate del linguaggio e le librerie utilizzate:

- **Utilizzo della libreria grafica JavaFX (AnimationTimer e Canvas)**
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/master/src/main/java/it/unibo/kikistore/engine/impl/GameEngineImpl.java#L47-L69>
- **Pattern Matching per instanceof**
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/master/src/main/java/it/unibo/kikistore/model/item/impl/ItemSpawnerImpl.java#L149-L156>
- **Utilizzo di Switch Expressions**
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/master/src/main/java/it/unibo/kikistore/view/entity/impl/EntityRenderer.java#L46-L52>
- **Lettura e parsing di risorse testuali con Java I/O**
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/master/src/main/java/it/unibo/kikistore/model/map/impl/MapLoader.java#L28-L51>
- **Collezioni non modificabili e copie (List.copyOf e List.of)**
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/master/src/main/java/it/unibo/kikistore/model/item/impl/ItemSpawnerImpl.java#L70>
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/master/src/main/java/it/unibo/kikistore/view/states/village/impl/VillageViewImpl.java#L96>
- **Uso esplicito di Iterator con rimozione sicura**
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/master/src/main/java/it/unibo/kikistore/model/item/impl/ItemSpawnerImpl.java#L145-L160>

Risorse Grafiche

Tutti gli sprite utilizzati per la mappa e l'ambiente di gioco della mia parte provengono dal progetto open-source *LPC Terrain Repack*:

- **Fonte:** OpenGameArt – *LPC Terrain Repack*
<https://opengameart.org/content/lpc-terrain-repack>

3.2.2 Hafsa Nouij

Vengono elencate di seguito le funzionalità avanzate del linguaggio e le librerie utilizzate:

- **Progettazione di metodi generici con tipi bounded** (`<T extends GameItem>`)
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/93113ca8423ec2a388377735c1a5714c0a4b9c91/src/main/java/it/unibo/kikistore/controller/impl/InventoryControllerImpl.java#L61-L69>
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/93113ca8423ec2a388377735c1a5714c0a4b9c91/src/main/java/it/unibo/kikistore/controller/impl/InventoryControllerImpl.java#L82-L94>
- **Uso di lambda expressions e method reference con interfacce funzionali di libreria** (`Supplier`, `Consumer`)
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/93113ca8423ec2a388377735c1a5714c0a4b9c91/src/main/java/it/unibo/kikistore/controller/impl/InventoryControllerImpl.java#L126-L148>
- **Uso di Optional** (con `map` e `orElse`)
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/93113ca8423ec2a388377735c1a5714c0a4b9c91/src/main/java/it/unibo/kikistore/controller/impl/RecipeBookControllerImpl.java#L46-L54>
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/93113ca8423ec2a388377735c1a5714c0a4b9c91/src/main/java/it/unibo/kikistore/engine/state/CraftingState.java#L303-L309>
- **Uso di Stream**
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/93113ca8423ec2a388377735c1a5714c0a4b9c91/src/main/java/it/unibo/kikistore/controller/impl/CraftingControllerImpl.java#L64-L69>
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/93113ca8423ec2a388377735c1a5714c0a4b9c91/src/main/java/it/unibo/kikistore/controller/impl/InventoryControllerImpl.java#L155-L160>
- **Utilizzo della libreria Gson per il caricamento dei dati di gioco da JSON**
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/93113ca8423ec2a388377735c1a5714c0a4b9c91/src/main/java/it/unibo/kikistore/model/inventory/impl/JsonResources.java#L29-L39>
- **Utilizzo della libreria JavaFX** (effetti grafici `ColorAdjust` per gli oggetti non posseduti)
Permalink: <https://github.com/Leon7-cmd/00P25-Kiki-Store/blob/93113ca8423ec2a388377735c1a5714c0a4b9c91/src/main/java/it/unibo/kikistore/view/book/InventorySection.java#L212-L216>

Risorse Grafiche

Gli sprite utilizzati per le pozioni e per gli ingredienti sono stati presi da un *Alchemy Pack* di sprite su un sito web di distribuzione di asset digitali bidimensionali (2D) per lo sviluppo di videogiochi - distribuiti con licenze d'uso gratuite.

- **Fonte:** craftpix – *Magic book asset pack*
<https://craftpix.net/freebies/free-animated-magic-book-pixel-art-asset-pack/>

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Leonardo Galimi

Autovalutazione

Uno dei risultati più soddisfacenti dell'intero lavoro è il sistema grafico e di gestione del mondo. L'idea di partenza prendeva ispirazione dalla gestione a tilemap e schermate di *Game Maker Studio*, in cui la griglia, il posizionamento degli elementi e il ciclo di disegno sono meccanismi già pronti. In questo caso, i **GameState** svolgono un ruolo ancora più grande rispetto alle *room*: non gestiscono solo lo scenario, ma l'intero contesto logico e operativo (ciclo di vita con sospensione e ripresa, controller dedicati e visualizzazione). Riuscire a progettare e implementare da zero questi processi, curando la logica delle matrici, il rilevamento delle collisioni, la telecamera e i renderers su **Canvas**, ha rappresentato una sfida stimolante.

Il punto di forza di questa implementazione sta nella sua scalabilità: il motore consente di importare nuove mappe, aggiungere ulteriori tipologie di oggetti e gestire altre entità semplicemente seguendo le interfacce e i contratti definiti.

Lavori futuri

L'architettura adottata consente di progettare diverse estensioni per rendere l'esperienza più ricca e completa:

- **Sistema di salvataggio persistente:** implementare un meccanismo per la serializzazione su file (ad esempio in formato JSON) per salvare lo stato della sessione, così da permettere una continuità di gioco tra le varie partite.
- **Mondo e viaggio in volo:** espandere l'esplorazione collegando più villaggi su isole differenti. Sfruttando la scalabilità del motore delle mappe, il volo potrebbe diventare un meccanismo di viaggio per spostarsi tra un'isola e l'altra, introducendo ingredienti esclusivi e nuove ricette specifiche per ciascun nuovo villaggio.

4.1.2 Hafsa Nouij

Autovalutazione

All'interno del progetto mi sono occupata dell'inventario, del ricettario, della preparazione delle pozioni, del minigioco del Memory e della definizione dei dati di gioco (ingredienti, pozioni e ricette) nei file JSON, curando per ciascuna parte model, controller e schermata. Ho inoltre disegnato in pixel art lo sprite della protagonista, Kiki.

Sono particolarmente soddisfatta del libro dell'inventario e del ricettario, con le sue animazioni di apertura e di cambio pagina, e del calderone, che rende la preparazione delle pozioni un momento di gioco vero e proprio. Anche il Memory mi ha dato soddisfazione: è un minigioco completo, con punteggio, ricompense e la possibilità di rigiocare. Dal punto di vista del codice, sono contenta di aver tenuto separata la logica di gioco dalle schermate: questo mi ha permesso di testare inventario, crafting e Memory senza l'interfaccia grafica, e i test mi hanno aiutata a scoprire errori di integrazione che non emergevano provando le parti singolarmente.

Tra gli aspetti che migliorerei, le tre schermate che si aprono sopra il villaggio (libro, calderone e Memory) condividono parte del codice: caricamento dei font, disegno dello sfondo e chiusura. Ho valutato di estrarre una classe base comune con il pattern Template Method, ma ho scelto di non introdurla a ridosso della consegna, per non rischiare regressioni dopo l'integrazione con il codice del compagno, dato che la duplicazione è limitata. Inoltre, alcune regole dell'inventario potrebbero stare nel model anziché nel controller.

Lavori futuri

- **Classe base per le schermate sovrapposte:** introdurre il pattern Template Method per definire una sola volta lo scheletro comune di libro, calderone e Memory.
- **Sezione degli ordini:** completare la terza sezione del libro, già prevista tra i segnapagina, con gli ordini dei clienti, da soddisfare con le pozioni preparate cercandole in base all'effetto richiesto e da consegnare direttamente alle loro casette nel villaggio.
- **Inventario più robusto:** spostare le regole sulle quantità nel model e rendere immutabili ingredienti e pozioni.

4.2 Difficoltà incontrate e commenti per i docenti

4.2.1 Leonardo Galimi

Essendo la mia prima esperienza su un progetto software di queste dimensioni — avendo affrontato in precedenza soltanto piccoli elaborati alle superiori —, la sfida principale non è stata solo tecnica, ma soprattutto organizzativa.

Nel corso dello sviluppo è emerso quanto sia stato un errore sottovalutare la fase preliminare: la mancanza di scadenze intermedie chiare, di ruoli ben definiti e di una comunicazione costante all'interno del team ha finito per pesare sull'intero lavoro, portando a rallentamenti e disallineamenti che si sono fatti sentire fino alle fasi finali.

A livello personale, faticando a gestire la pressione e le decisioni affrettate a ridosso della consegna, questa esperienza mi ha insegnato quanto sia indispensabile partire con il piede giusto: stabilire bene le responsabilità sin dal primo giorno non è una formalità, ma l'unico modo per tenersi la testa libera dallo stress e riuscire a ragionare su quello che si sta programmando.

4.2.2 Hafsa Nouij

La difficoltà principale è stata organizzativa. Durante lo sviluppo il gruppo è passato da tre a due componenti, e riorganizzare il lavoro e integrare le parti sviluppate separatamente ha richiesto più tempo del previsto, portando a una consegna più tarda di quanto avessimo pianificato.

Ho vissuto questo periodo con molto stress. Il ritardo si è sovrapposto agli altri impegni, e a questo si è aggiunta la pressione che mi mettevo da sola: mi ero posta obiettivi molto ambiziosi per la mia parte, e questo ha reso tutto più pesante di quanto sarebbe dovuto essere. Ripensandoci, una comunicazione più chiara all'interno del gruppo fin dall'inizio avrebbe aiutato parecchio.

Da questa esperienza mi porto a casa che gli imprevisti capitano e vanno affrontati insieme al gruppo, e che darmi obiettivi più realistici mi aiuterebbe a lavorare con meno stress.

Appendice A

Guida utente

A.1 Inizio

All'avvio dell'applicativo l'utente si trova davanti a questa schermata di menu (Figura A.1)



Figura A.1: Schermata Iniziale

La schermata mostra due bottoni cliccabili con il mouse, posizionati nell'angolo in basso a sinistra:

- **GIOCA:** avvia la sessione di gioco.
- **ESCI:** termina l'esecuzione dell'applicazione.

A.2 Controlli di gioco

Il gioco è controllabile principalmente da tastiera. La tabella seguente riassume i comandi principali:

Tasto	Azione
W, A, S, D / Freccie direzionali	Movimento del personaggio (Su, Sinistra, Giù, Destra)
E	Interazione / Conferma selezione
I	Apertura dell'inventario
C	Apertura del crafting
ESC	Pausa di gioco / Uscita dalle schermate

Tabella A.1: Mappatura dei controlli da tastiera.

A.3 Dinamiche di gioco

A.3.1 Villaggio ed Esplorazione

Dopo aver avviato la partita, il giocatore si trova all'interno del villaggio (Figura A.2), l'ambiente principale del gioco dove si può esplorare e interagire con il mondo di gioco.



Figura A.2: Schermata di esplorazione del villaggio.

All'interno del villaggio l'utente può eseguire diverse azioni:

- **Raccolta degli ingredienti:** avvicinandosi agli oggetti sparsi sul terreno, il player li raccoglie automaticamente e vengono aggiunti all'inventario;
- **Accesso ai minigiochi:** interagendo con le aree di teletrasporto nella mappa, è possibile accedere ai minigiochi.

A.3.2 Inventario e ricettario

In questa schermata possiamo visualizzare la sezione relativa all'inventario e quella per il ricettario (Tasto "I")

Qui è possibile visualizzare la quantità disponibile di ogni item (Figura A.3)



Figura A.3: Schermata d'inventario.

Qui invece possiamo visualizzare tutte le ricette che abbiamo sbloccato. (Figura A.4)



Figura A.4: Schermata del ricettario.

L'intera interfaccia si naviga usando solamente le frecce direzionali:

- Frecce **Su** / **Giù**: permettono di scorrere tra le diverse sezioni.
- Frecce **Destra** / **Sinistra**: permettono di sfogliare le pagine, nel caso del ricettario.

A.3.3 Crafting

Il crafting è la schermata che consente la creazione delle pozioni (Tasto "C")



Figura A.5: Schermata di crafting.

Ogni pozione craftata con successo, riduce di 1 unità l'energia del player. Se il player non ha energia non può craftare. All'interno della schermata ci si muove usando le "freccie direzionali" e per selezionare gli ingredienti si usa il tasto "E"

A.3.4 Minigame

Per accedere ai minigame basta interagire (tasto "E") con le aree di teletrasporto indicate in figura



Figura A.6: Zone di teletrasporto nella mappa.